Selector Syntax

Earlier I gave some examples of CSS selectors and showed how `div > p` would match all `p` elements that are direct children of any `div` element: `div > p` is equivalent to XPath `/descendant-or-self::div/child::p` (abbreviates to `//div/p`). CSS selector expressions are roughly equivalent to XPath's, you can think of them as localization steps, where the leftmost step is closest to the root of the document (the document element) and, as we progress to the right of the expression, we descend the DOM tree. The precise syntax rules of CSS selectors are different from XPath. Sufficiently so that it's worth defining them before we do any deep comparison.

Selector Structure

The basic structure of a CSS selector is this: "simple" selectors can be combined into "compound" selectors to act on single elements. These simple/compound sequences are (fairly) equivalent to XPath localization steps. Simple/compound selectors can be further joined with *combinators* to form a "complex" selector sequence, just as XPath localization steps are joined together to form an XPath expression.

A simple CSS selector is a single instance of one of four basic selectors: `basic`, `attribute`, `pseudo-class`, and `pseudo-element`.

Table 11-19. *CSS selector basic types*

Simple type	Detail	Example
`basic`	Asterisk (aka "universal"), element name (aka "type"), class, id	`* div .class #id`
`attribute`	Any attribute (including `class` and `id` if you wish)	`[id] [href~="php"]`
`pseudo-class`	Matches state of an element	`:visited :paused :checked :has`
`pseudo-element`	Matches a part of an element	`::first-letter ::selection`

Some of these simple selectors don't make much sense in the context of PHP's DOM …, for example, many of the pseudo-class selectors rely on things like UI events